

Designing a Secure, Highly Available Cloud Solution for A Medical Company

A Proposed Amazon Web Services Architecture

ITC 6480 — Cloud Architecting

Final Project (Individual)

Author: Wisdom Djam

Major: M.P.S. Informatics
(Information Security Management)

Institution: Northeastern University

Date: June 23, 2026

Prepared for the customer “A Medical Company” as a design proposal.

1 Introduction

A Medical Company is a medical software-as-a-service (SaaS) startup whose platform connects patients and doctors for online appointments, remote consultation and diagnosis, electronic prescription transfer, payment processing, and document and image handling. The company serves users across the Asia-Pacific, United States, and European markets, and it is preparing for a public launch with the expectation of rapid and unpredictable growth in its user base. Because the platform sits between patients and clinicians, the volume and sensitivity of the data it processes are both high: it stores and transmits medical personally identifiable information (PII), prescription records, and payment data, all of which carry legal and ethical obligations for confidentiality, integrity, and availability.

The problem this project addresses is that the company's current development and test environment runs on leased physical Windows servers at a third-party hosting provider. That environment uses Internet Information Services (IIS) for its web and application tiers and Microsoft SQL Server for its database. While adequate for early development, this arrangement is unsuitable for a public production launch for several reasons. First, it cannot scale on demand: a fixed number of physical servers cannot absorb the traffic spikes expected at launch or the steady growth that follows, which risks slow response times or outages precisely when first impressions matter most. Second, it contains single points of failure: with servers concentrated in one location, the failure of a server, a rack, or the hosting facility itself could take the entire platform offline. Third, it was not designed for the strict security, encryption, and auditability that medical PII demands, leaving the company exposed to both regulatory and reputational risk.

This report proposes a migration to Amazon Web Services (AWS) that resolves each of these problems. The proposed solution is a multi-tier, multi-Availability Zone architecture that preserves the company's familiar three-tier design while adding four capabilities the current environment lacks: elasticity, so capacity follows demand automatically; high availability, so the platform survives the loss of an individual component or an entire Availability Zone; defense-in-depth security, so medical PII is protected at every layer; and complete auditing, so every action taken in the environment is recorded. The design is grounded in the AWS Well-Architected Framework, which organizes cloud best practices into five pillars — operational excellence, security, reliability, performance efficiency, and cost optimization — and provides a consistent basis for evaluating architectural decisions (Amazon Web Services, n.d.-a). The remainder of this report presents the requirements and assumptions that shaped the design, the solution organized by area with its supporting worksheets and screenshots from a working AWS environment, and a recommended path to production.

2 Executive Summary

The objective of this project was to translate the customer’s seven stated requirements into a concrete AWS reference architecture that mirrors their current three-tier topology while removing every single point of failure. The work was completed as an individual project, and the design decisions were verified hands-on in a live AWS account so that the proposal reflects real, configurable services rather than theory alone.

The recommended solution can be summarized across six areas. For **compute**, Windows-based Amazon Elastic Compute Cloud (EC2) instances host the web and application tiers inside Auto Scaling Groups that span two Availability Zones, fronted by Elastic Load Balancing. For the **database**, Amazon Relational Database Service (RDS) for SQL Server runs in a Multi-AZ configuration with a synchronous standby and automatic failover. For **networking**, the environment is divided into two isolated virtual private clouds (VPCs) — one for production and one for development and test — with public subnets reserved for load balancers and private subnets for the application and database tiers. For **identity and access**, AWS Identity and Access Management (IAM) enforces least-privilege groups, multi-factor authentication, and application roles, backed by a strong account password policy. For **resilience**, Elastic Load Balancing and Auto Scaling provide self-healing of the web and application tiers, while Multi-AZ RDS protects the database. For **governance**, AWS CloudTrail records all user activity for auditing and Amazon CloudWatch provides monitoring and the alarms that drive scaling, with logs retained in a secured Amazon Simple Storage Service (S3) bucket.

Each of these choices maps directly to one or more customer requirements, ensuring the architecture is requirement-driven. The overall recommendation is to proceed with this design and to validate it through a proof of concept before the public launch. Because the application handles medical data, the design assumes the use of HIPAA-eligible AWS services and a signed AWS Business Associate Addendum, with formal compliance validation scheduled as part of that proof of concept. Figure 1 shows the AWS account in which the design was configured.

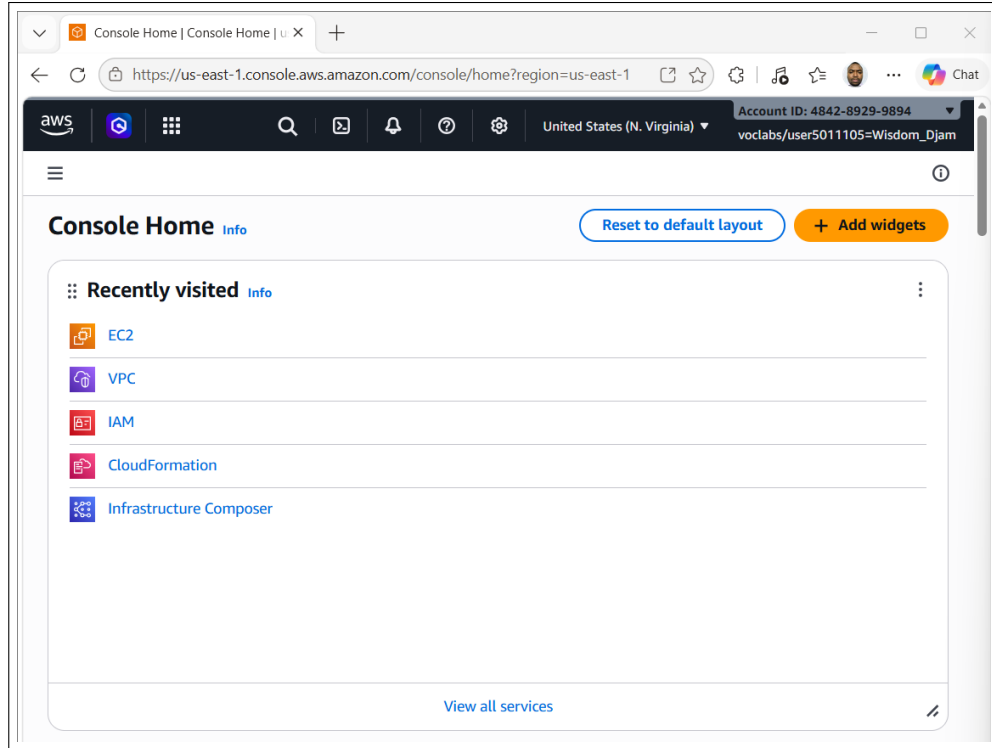


Figure 1: The AWS Console Home for the account used to build and verify the design, showing the services exercised during the project (EC2, VPC, IAM, and CloudFormation).

3 Overall Requirements and Assumptions

3.1 Requirements

The design was driven by the customer’s requirements, derived from their high-level goals and detailed network constraints. First, access permissions must conform to AWS best practices, applying the principle of least privilege together with multi-factor authentication and a strong password policy. Second, the networks must follow best practice and provide all the services needed across each environment. Third, the architecture must match the company’s current three-tier design and be able to double the number of servers on demand to absorb growth. Fourth, all medical PII must be secured through isolation, controlled access, and encryption. Fifth, the web and application tiers each require load balancers supporting HTTP, HTTPS, and TCP traffic. Sixth, the architecture must be resilient and built for business continuity. Seventh, an auditing capability must track all user actions across the environment.

In addition, the customer specified several network constraints. There must be no external (internet) access to the application or database tiers; IP address usage should be minimized to reduce the attack surface; the development and test environment must be separated from production; the public web load balancer should receive internet traffic on port 443; and each tier should accept traffic only from the tier in front of it, on port 443.

3.2 Assumptions

Several assumptions made the design tractable and defensible. The application is assumed to be compatible with AWS-hosted Windows Server and Amazon RDS for SQL Server, allowing a lift-and-shift migration with minimal refactoring. A single AWS Region (for example, US East) is assumed acceptable for the initial launch, with multi-Region disaster recovery treated as a later phase, and two Availability Zones are assumed sufficient for high availability at launch. The customer is assumed to hold a Transport Layer Security (TLS) certificate, provisioned through AWS Certificate Manager, for HTTPS termination at the load balancer. The requirement to double the servers is satisfied by configuring the Auto Scaling Group's maximum capacity rather than by manual provisioning. Security, availability, and compliance are prioritized above cost for the initial design, with cost optimization addressed in a later phase. Finally, because the platform stores medical data, the design assumes the use of HIPAA-eligible AWS services and an executed AWS Business Associate Addendum, and it assumes the database administrators require only standard access (no advanced relational-database configuration), which makes a managed database service the natural choice.

4 Solution

The solution is presented in the sections below. Each section summarizes the relevant requirements, presents the required worksheet, and explains the reasoning behind the chosen approach.

4.1 Solution — Identify AWS Services

Requirements overview. The architecture must provide compute, database, networking, identity, storage, and governance services that together satisfy all seven customer requirements while following AWS best practices.

Identify AWS Services Worksheet.

Need	AWS Service	Purpose in the design
Compute (web & app)	Amazon EC2 + EC2 Auto Scaling	Windows web/app instances that scale across two AZs
Load balancing	Elastic Load Balancing (ALB)	Distribute traffic; health checks for self-healing
Database	Amazon RDS for SQL Server (Multi-AZ)	Managed, highly available relational database
Networking	Amazon VPC, subnets, security groups, Internet Gateway	Isolated network with public/private separation
Identity & access	AWS IAM (groups, roles, MFA, password policy)	Least-privilege access control
Object storage	Amazon S3	Uploaded documents/images and log storage
Auditing	AWS CloudTrail	Record of all user/API actions
Monitoring	Amazon CloudWatch	Metrics, alarms, and scaling triggers
Encryption	AWS Certificate Manager + AWS KMS	TLS in transit; encryption at rest

Why this approach. The service selection deliberately favors AWS managed services over self-managed equivalents. Managed services such as RDS and Elastic Load Balancing reduce the operational burden on a small startup team while providing built-in high availability, which directly supports the reliability pillar of the Well-Architected Framework (Amazon Web Services, n.d.-a). Each service maps to a specific requirement, ensuring the architecture is requirement-driven rather than arbitrary.

4.2 Solution — User Authentication

Requirements overview. Access permissions must follow AWS best practices: least privilege, multi-factor authentication, role-based access for applications, and a strong password policy (Requirement 1).

User Authentication Worksheet.

IAM Entity	Permissions (least privilege)	Authentication
SysAdmin group	Manage EC2, VPC, Auto Scaling, ELB	Console + MFA required
DBA group	Manage / connect to RDS (standard access)	Console + MFA required
Monitoring group	Read-only on EC2, S3, RDS, CloudWatch	Console + MFA required
Application role	Read/write to specific S3 bucket only	IAM role (temporary credentials; no stored keys)
Account password policy	8-char min.; upper, lower, number, special; 90-day rotation; no reuse of last 3	Applies to all IAM users

Why this approach. Grouping users and attaching least-privilege policies to groups rather than to individuals is a core AWS IAM best practice: it simplifies administration, ensures consistency, and limits the blast radius of any single compromised credential (Amazon Web Services, n.d.-b). The three groups reflect a separation of duties appropriate to a medical platform — system administrators, database administrators, and a monitoring function that can observe but not change resources. Using an IAM role for the application, rather than embedding access keys in code, means the application receives temporary, automatically rotated credentials and there are no long-lived secrets to leak. Requiring multi-factor authentication for all human console access adds a second factor beyond the password, which materially reduces the risk of account takeover. Finally, the account password policy enforces the length, complexity, rotation, and reuse rules expected for a system handling sensitive health information. Figure 2 shows this policy configured in the AWS console.

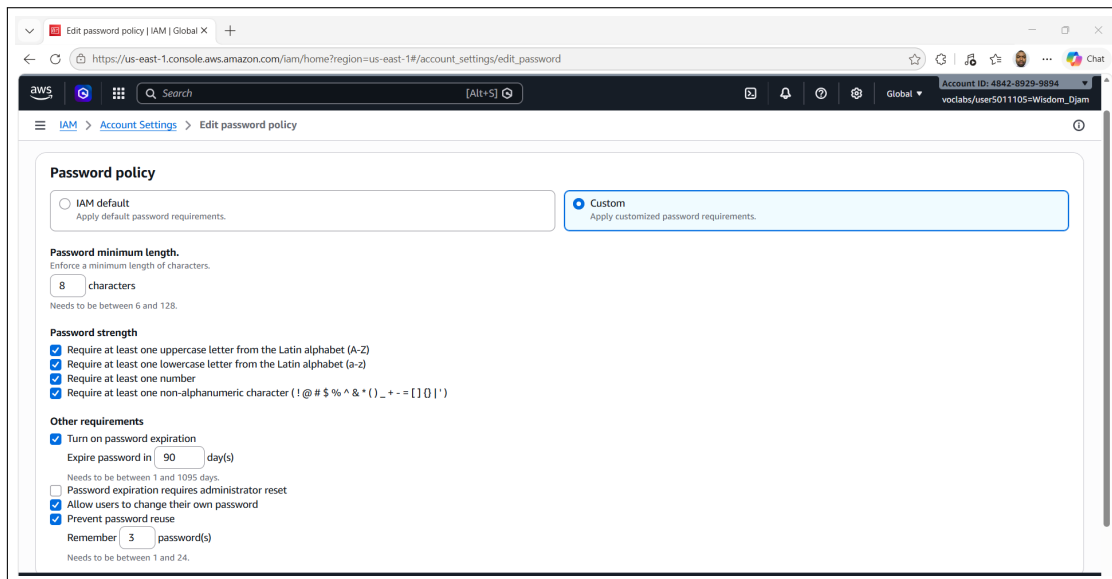


Figure 2: The IAM account password policy: an eight-character minimum, all four complexity classes, 90-day expiration, and prevention of reuse of the last three passwords. (In the AWS Academy lab the assumed role has read-only IAM permissions, so the policy is shown as configured; in a production account with the appropriate permission it would be committed — an illustration of how IAM permission boundaries constrain a role.)

4.3 Solution — Network and Security

Requirements overview. The network must isolate environments, prevent external access to the application and database tiers, restrict tier-to-tier traffic to port 443, minimize IP usage, and secure all medical PII (Requirements 2 and 4).

Network and Security Worksheet.

Tier / Component	Subnet type	Security group inbound	Rationale
Web ELB	Public	443 from the Internet	Only public entry point
Web EC2	Private	443 from Web ELB SG	Not reachable from internet
App ELB	Private	443 from Web tier SG	Internal traffic only
App EC2	Private	443 from App ELB SG	Not reachable from internet
RDS (SQL Server)	Private	443 from App tier SG only	No internet path to the database

Why this approach. Security groups are configured as a chain in which each tier accepts traffic only from the tier directly in front of it, on port 443. This implements defense in depth at the network layer: even if one tier were compromised, an attacker could not reach the database directly, because the database's security group references the application tier's security group as the only permitted source, rather than any IP range. This is a stronger control than an address-based rule, because it remains correct even as instances are added or replaced by Auto Scaling and their IP addresses change. Placing the web, application, and database tiers in private subnets with no route to the Internet Gateway eliminates external access to those tiers entirely, leaving the public web load balancer as the single, deliberately hardened entry point and thereby minimizing the attack surface. Using two separate VPCs prevents development and test activity from ever reaching production patient data and contains the impact of any misconfiguration to a single environment. Finally, encryption is applied in transit with TLS through AWS Certificate Manager and at rest using AWS Key Management Service across the EBS volumes, the RDS database, and the S3 buckets, protecting medical PII throughout its lifecycle (Amazon Web Services, n.d.-c). Figure 3 shows the database-tier security group enforcing this design.

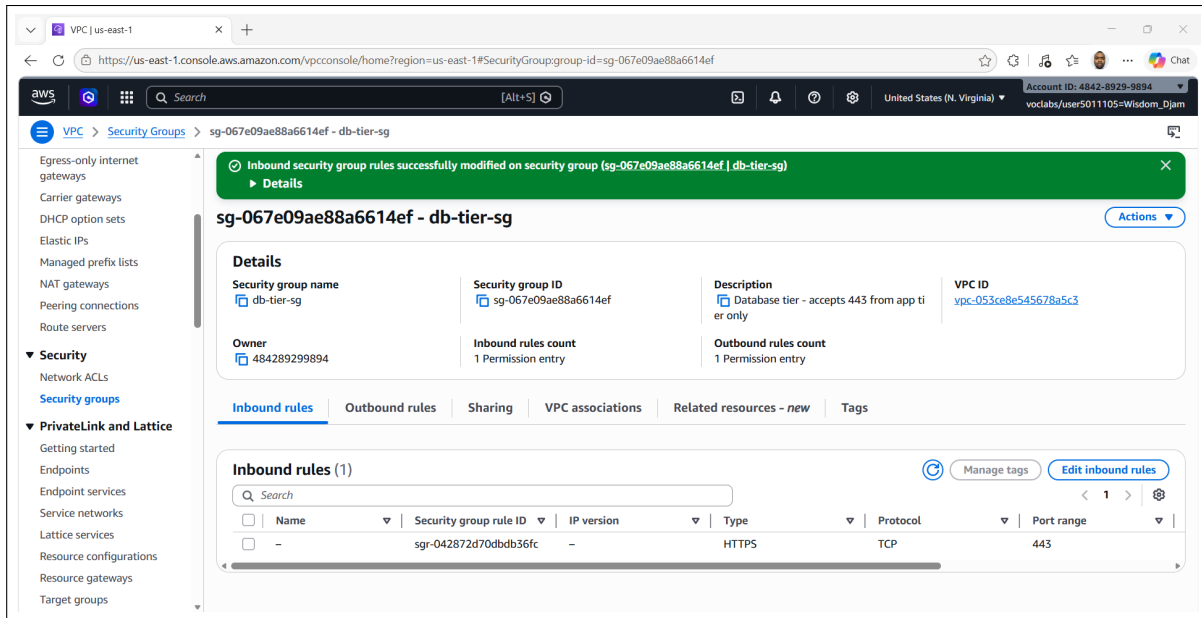


Figure 3: The database-tier security group (db-tier-sg) inbound rule: HTTPS on port 443 with the source set to the application-tier security group (app-tier-sg) — a security group reference, not an IP range. The database therefore accepts traffic only from the application tier.

4.4 Solution — Web and Application Tier (Architecture Diagram)

Requirements overview. The architecture must match the existing three-tier design, provide load balancers for the web and application tiers, and span multiple Availability Zones (Requirements 3 and 5).

Why this approach. The three-tier topology mirrors the customer’s existing design, minimizing application changes during migration. Two load balancers — a public-facing web load balancer and an internal application load balancer — keep the application tier private while still distributing traffic, and spreading every tier across two Availability Zones removes single-zone failure as a single point of failure (Amazon Web Services, n.d.-d).

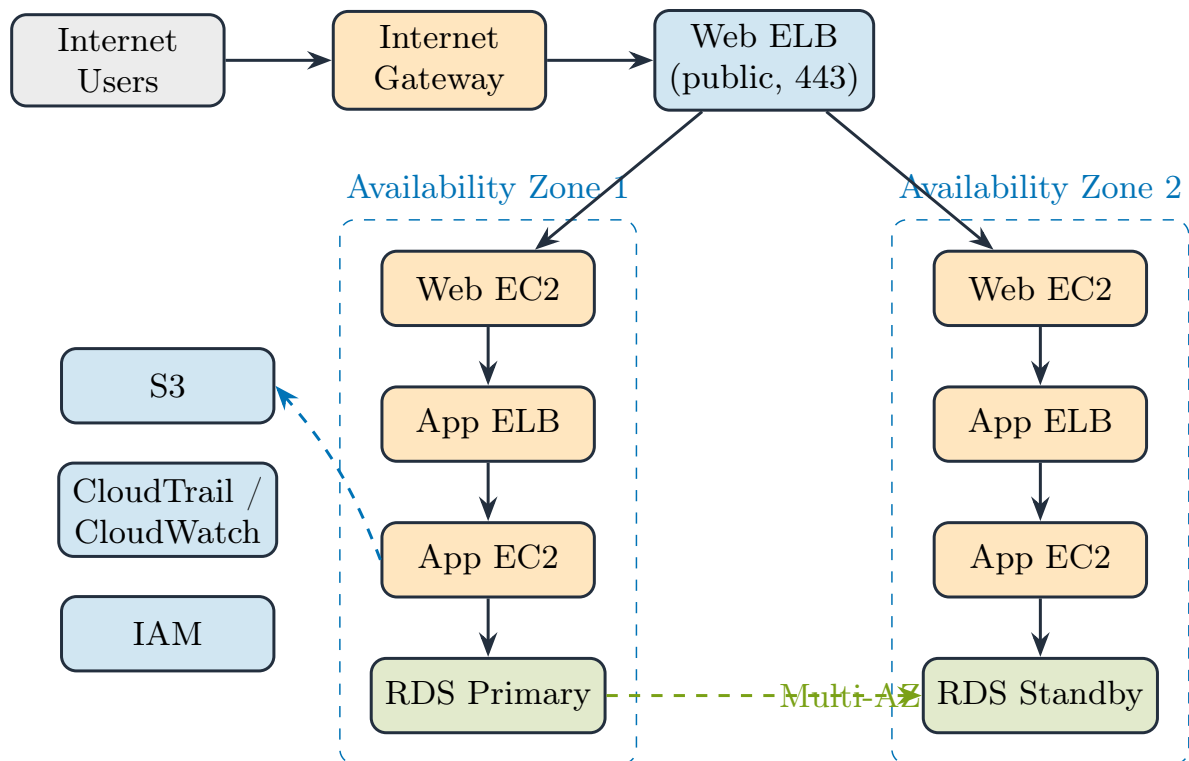


Figure 4: Production VPC architecture across two Availability Zones. The public Web ELB is the only internet-facing component; web, application, and database tiers reside in private subnets. RDS runs Multi-AZ with a synchronous standby. A mirror VPC (not shown) hosts the isolated Development/Test environment.

4.5 Solution — Web and Application Tier (Instance Justification)

Requirements overview. The web and application tiers must scale to handle launch demand and the ability to double capacity (Requirement 3).

Instance Justification Worksheet.

Tier	Tag	OS	Scaling	Justification
Web	Key=Name, Value=web-tier	Windows Server	ASG (Min 2 / Max 6)	Stateless web servers; scale on CPU
App	Key=Name, Value=app-tier	Windows Server	ASG (Min 2 / Max 6)	Business logic tier; scale on CPU

DB	Key=Name, Value=db-tier	Managed (RDS)	Multi- AZ	Managed SQL Server; automatic failover
----	----------------------------	------------------	--------------	---

Why this approach. Placing the web and application tiers in Auto Scaling Groups with a minimum of two and a maximum of six instances satisfies the requirement to double the servers on demand: growth becomes a configuration value rather than a hardware-procurement project. A minimum of two instances per tier (one per Availability Zone) ensures the tier remains available even if an entire zone fails. The managed RDS database removes the need to size and maintain database servers manually.

4.6 Solution — Business Continuity

Requirements overview. The architecture must be resilient and provide business continuity, recovering automatically from component and Availability Zone failures (Requirement 6).

Business Continuity Worksheet.

Failure scenario	Mechanism	Outcome
EC2 instance fails	ELB health check + Auto Scaling	Unhealthy instance replaced automatically
Availability Zone fails	Resources span two AZs	Healthy AZ continues serving traffic
Database primary fails	RDS Multi-AZ standby	Automatic failover to standby
Traffic spike (launch)	Auto Scaling scale-out	Capacity added up to the maximum

Why this approach. Elastic Load Balancing and Auto Scaling together create a self-healing web and application tier. The load balancer continuously runs health checks against the instances behind it; when an instance fails a check — because the operating system or the application has stopped responding — the load balancer stops routing traffic to it, and the Auto Scaling Group terminates the unhealthy instance and launches a fresh replacement from the launch template, all without human intervention. Because the Auto Scaling Group spans two Availability Zones with a minimum of two instances, the loss of an entire zone still leaves a healthy instance serving traffic in the other zone. Amazon RDS Multi-AZ extends this resilience to the database by maintaining a synchronous standby replica in a second Availability Zone and failing over to it automatically if the primary becomes unavailable, so the database is not a single point of failure (Amazon Web Services, n.d.-d). It is worth distinguishing the two mechanisms the design relies upon, because they are often confused: load balancing distributes current traffic across the servers that already exist, whereas elasticity changes how many servers exist over time. Load balancing alone cannot absorb sustained growth, and scaling alone cannot evenly distribute a given moment's traffic; the architecture needs both, which is why the design pairs Elastic Load Balancing with Auto Scaling at each tier.

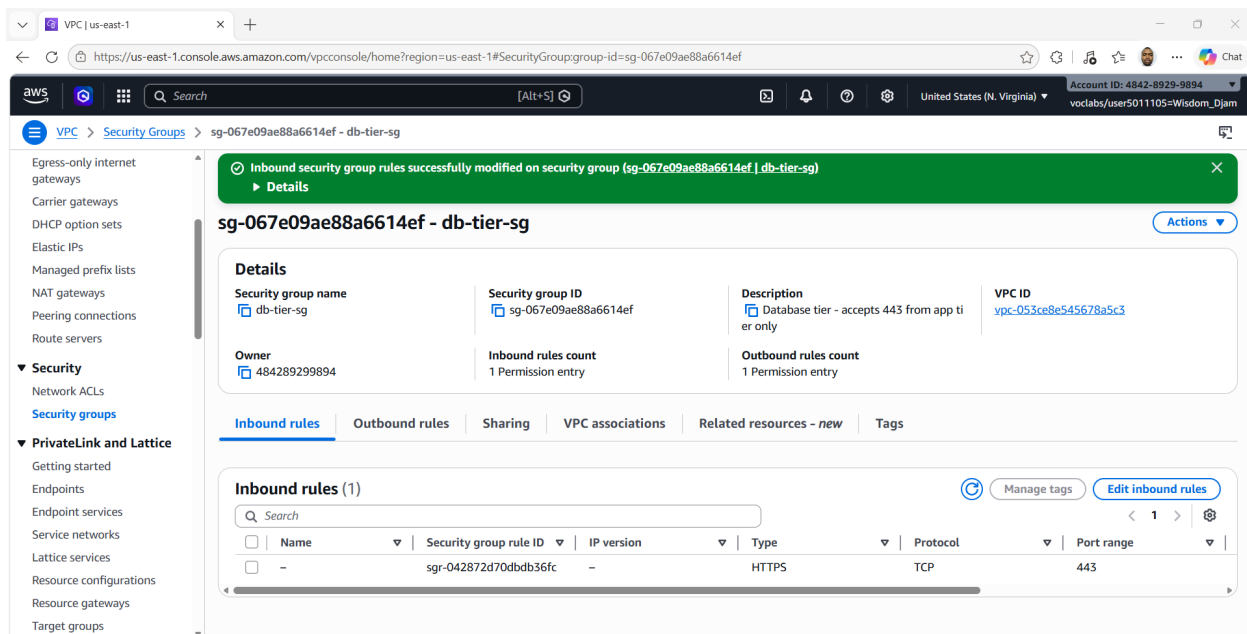


Figure 5: Enter Caption

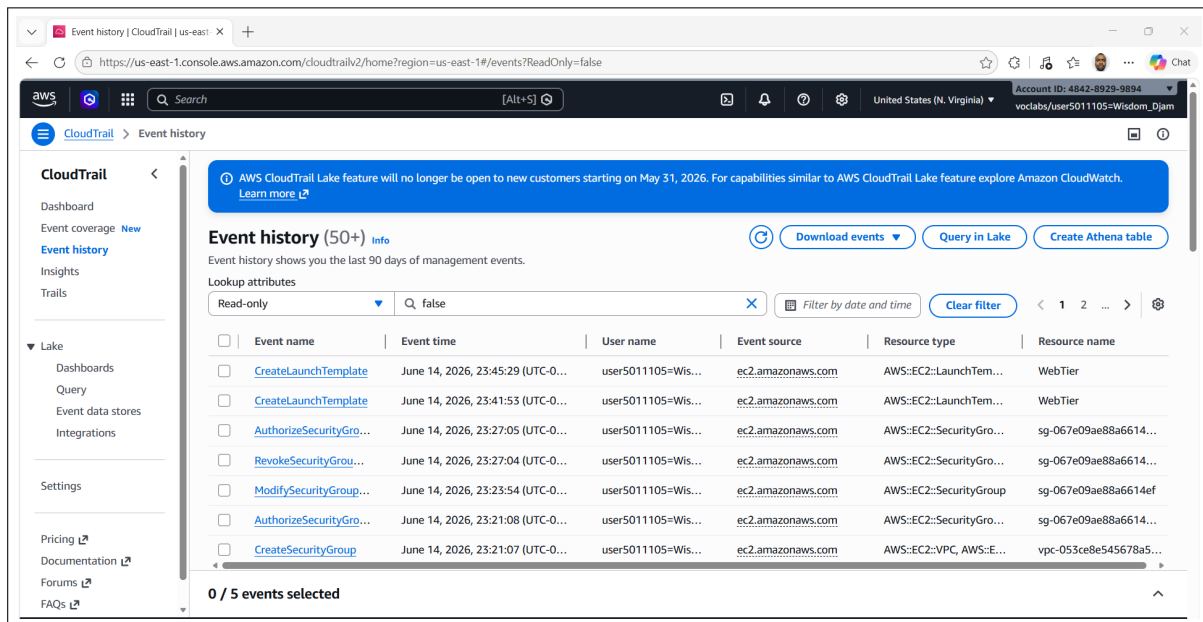
4.7 Solution — Auditing

Requirements overview. An auditing capability must track all user actions across the environment (Requirement 7).

Auditing Worksheet.

Capability	Service / configuration	Purpose
User/API action logging	AWS CloudTrail (all Regions)	Records who did what, when
Log storage	Dedicated, access-restricted S3 bucket; log-file validation enabled	Secure, tamper-evident records
Operational monitoring	Amazon CloudWatch metrics & alarms	Performance; triggers Auto Scaling
Separation of duties	Read-only Monitoring IAM group	View without modify rights

Why this approach. AWS CloudTrail records every API call made in the account — whether issued through the management console, the command-line interface, or a software development kit — capturing who performed each action, when, and against which resource. This provides the complete, attributable audit trail that the customer requires and that a medical platform needs in order to demonstrate accountability (Amazon Web Services, n.d.-e). Delivering those logs to a dedicated, access-restricted S3 bucket with log-file validation enabled makes the trail tamper-evident: if a log file were altered or deleted, validation would reveal it, which preserves the integrity of the record even against a malicious or careless administrator. Amazon CloudWatch complements CloudTrail by addressing a different question — not *who did what*, but *how is the system performing*. CloudWatch collects metrics, presents dashboards, and raises alarms; the same alarms that warn operators of high utilization are also what trigger the Auto Scaling actions described in the previous section, so monitoring and scaling form a single closed loop. A read-only Monitoring IAM group enforces separation of duties by allowing staff to observe resources without the ability to change them (Amazon Web Services, n.d.-f). Figure 9 shows real CloudTrail activity captured during this project, and Figure 10 shows a representative CloudWatch alarm.



The screenshot shows the AWS CloudTrail Event history page. A blue banner at the top states: "AWS CloudTrail Lake feature will no longer be open to new customers starting on May 31, 2026. For capabilities similar to AWS CloudTrail Lake feature explore Amazon CloudWatch." Below this, the "Event history (50+)" section shows a table of events. The table has columns for Event name, Event time, User name, Event source, Resource type, and Resource name. The events listed are:

Event name	Event time	User name	Event source	Resource type	Resource name
CreateLaunchTemplate	June 14, 2026, 23:45:29 (UTC-0...)	user5011105=Wis...	ec2.amazonaws.com	AWS::EC2::LaunchTem...	WebTier
CreateLaunchTemplate	June 14, 2026, 23:41:53 (UTC-0...)	user5011105=Wis...	ec2.amazonaws.com	AWS::EC2::LaunchTem...	WebTier
AuthorizeSecurityGro...	June 14, 2026, 23:27:05 (UTC-0...)	user5011105=Wis...	ec2.amazonaws.com	AWS::EC2::SecurityGro...	sg-067e09ae88a6614...
RevokeSecurityGrou...	June 14, 2026, 23:27:04 (UTC-0...)	user5011105=Wis...	ec2.amazonaws.com	AWS::EC2::SecurityGro...	sg-067e09ae88a6614...
ModifySecurityGroup...	June 14, 2026, 23:23:54 (UTC-0...)	user5011105=Wis...	ec2.amazonaws.com	AWS::EC2::SecurityGroup	sg-067e09ae88a6614ef
AuthorizeSecurityGro...	June 14, 2026, 23:21:08 (UTC-0...)	user5011105=Wis...	ec2.amazonaws.com	AWS::EC2::SecurityGro...	sg-067e09ae88a6614...
CreateSecurityGroup	June 14, 2026, 23:21:07 (UTC-0...)	user5011105=Wis...	ec2.amazonaws.com	AWS::EC2::VPC, AWS::E...	vpc-053ce8e545678a5...

At the bottom of the table, it says "0 / 5 events selected".

Figure 9: CloudTrail event history recording real user actions performed while building this design (for example, `CreateSecurityGroup` and `ModifySecurityGroupRules`), each with the user identity, timestamp, and affected resource — a live demonstration of the audit trail required by Requirement 7.

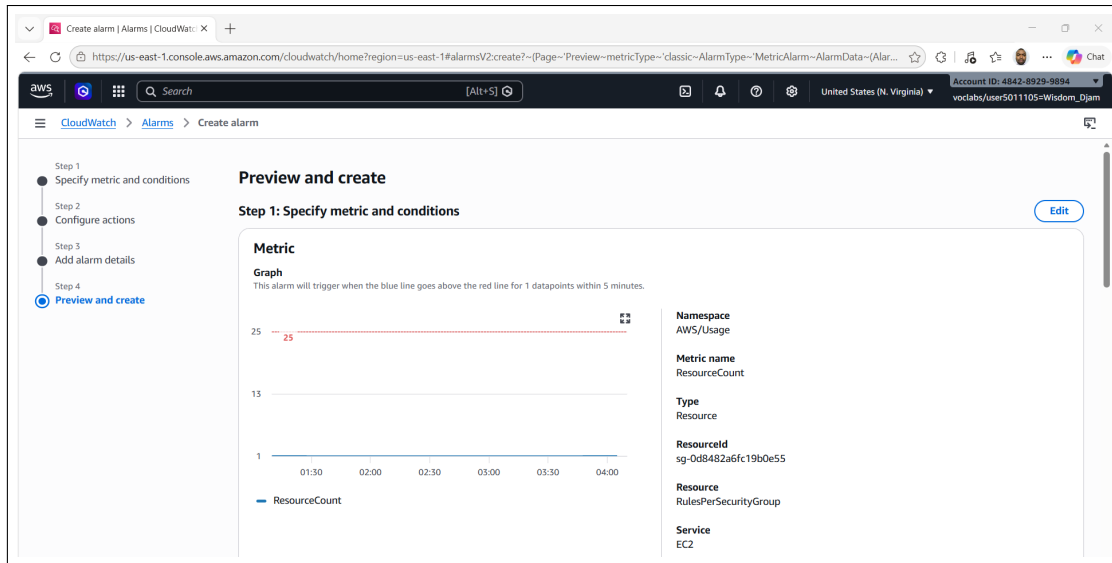


Figure 10: A CloudWatch alarm configured with a static threshold over a five-minute evaluation period. In production, a CPU-utilization alarm of this form is what triggers the Auto Scaling Group to add or remove instances. (The lab role lacks permission to commit the alarm, so it is shown configured; in production it would be created and wired to the scaling policy.)

5 Next Steps and Conclusion

The recommended next step is a proof of concept (POC). In the POC, the production VPC would be deployed in a single Region and subjected to load testing that deliberately stresses the architecture: instances would be terminated to confirm that the web and application tiers self-heal, and a database failover would be triggered to confirm that the RDS Multi-AZ standby takes over cleanly and within an acceptable recovery time. The POC would also exercise the Auto Scaling policies under simulated launch-day traffic to verify that capacity scales out and back in as intended. This validation de-risks the public launch by demonstrating, with evidence, that the resilience and scalability claims in this report hold in practice. The POC is also the appropriate point to formalize HIPAA alignment: signing the AWS Business Associate Addendum and conducting a review to confirm that every service touching PII is HIPAA-eligible, before any real patient data is loaded.

Beyond the POC, several enhancements are recommended for subsequent phases. First, multi-Region disaster recovery, with cross-Region replication of S3 backups, would protect the platform against the loss of an entire Region and is a natural progression once the single-Region design is proven. Second, a cost-optimization pass — applying Reserved Instances or Savings Plans to the steady-state compute baseline and right-sizing instances based on observed CloudWatch metrics — would reduce running costs once the launch-phase priority on security and availability can be balanced against efficiency. Third, end-user authentication for patients and doctors should be evaluated using Amazon Cognito or identity federation, which would provide managed sign-up, sign-in, and social or enterprise identity integration for the application’s own users, distinct from the administrative IAM access covered in this report.

In conclusion, the proposed architecture satisfies all seven of the customer’s requirements while adhering to AWS best practices and the Well-Architected Framework. It preserves the company’s familiar three-tier design, yet replaces a fragile, fixed-capacity environment with one that scales elastically, survives component and Availability Zone failures, restricts access to sensitive medical data through layered controls, and records every action for audit. The most valuable lesson from completing this project individually was that the central challenge of cloud architecting is not the technology itself but the translation of ambiguous business requirements into concrete services, and the clear communication of those choices to a non-technical customer. A related insight is that security, availability, and auditing are not isolated features to be bolted on at the end; they are cross-cutting concerns that shape decisions at every tier of the design. On the strength of this analysis, I recommend that A Medical Company proceed with the proposed architecture and validate it through the proof of concept described above.

References

- Amazon Web Services. (n.d.-a). *AWS Well-Architected Framework*. Retrieved June 23, 2026, from <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>
- Amazon Web Services. (n.d.-b). *Security best practices in IAM*. Retrieved June 23, 2026, from <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>
- Amazon Web Services. (n.d.-c). *What is Amazon VPC?* Retrieved June 23, 2026, from <https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html>
- Amazon Web Services. (n.d.-d). *Multi-AZ deployments for high availability (Amazon RDS)*. Retrieved June 23, 2026, from <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html>
- Amazon Web Services. (n.d.-e). *What is AWS CloudTrail?* Retrieved June 23, 2026, from <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-user-guide.html>
- Amazon Web Services. (n.d.-f). *What is Amazon CloudWatch?* Retrieved June 23, 2026, from <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html>